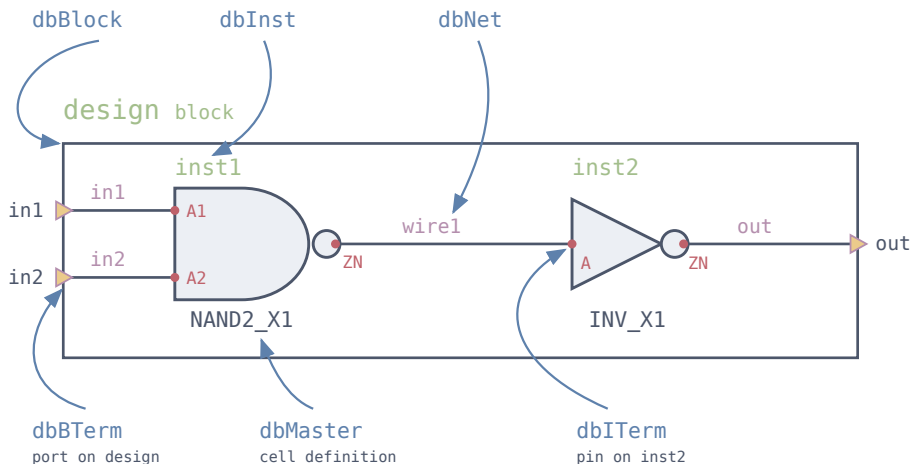


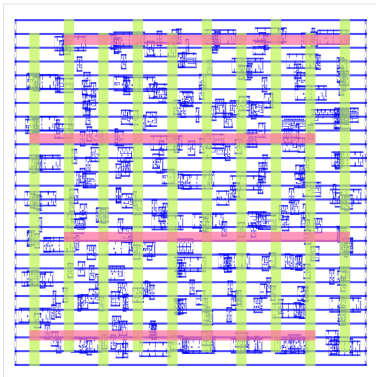
Ports (BTerms), cell pins (ITerms), wires (nets)



Four ways to do global placement

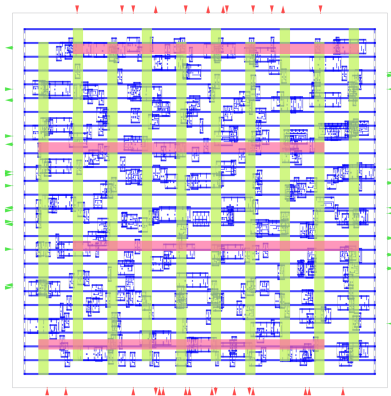
Mode	Pins	Needs
<code>global_placement</code>	Uses placed pins; <code>-skip_io</code> ignores them	Floorplan, cells, and rows
<code>-place_ios</code>	Co-optimizes movable pins and cells	Supported die and pin setup; run <code>place_pins</code> afterward
<code>-routability_driven</code>	Uses placed pins	Routing layers for congestion estimates; no prior routing or parasitic estimate
<code>-timing_driven</code>	Uses placed pins	Liberty, timing constraints, and wire RC; CTS is not required

Routing-aware placement setup



```
# Reserve 50% for later routing.  
# Keep placement routable.  
# Set signal and clock layer ranges.  
set_global_routing_layer_adjustment \  
    metal2-metal10 0.5  
set_routing_layers \  
    -signal metal2-metal10 \  
    -clock metal6-metal10
```

Final routability-driven placement



Placement mode alternatives:

-place_ios

Co-optimize pin placement with
cell placement.

-skip_io

Place cells without considering
or needing pin locations.

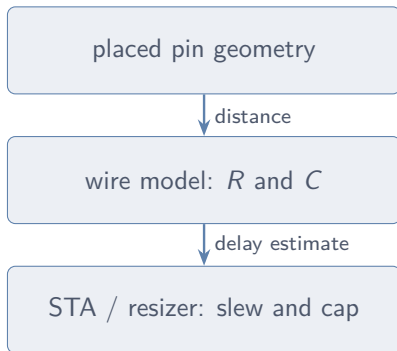
Density is between 0 and 1.

Default is 0.7.

Final pass: use congestion in the
objective after pins are placed.

```
global_placement \  
-routability_driven \  
-density 0.80 \  
-pad_left 2 -pad_right 2
```

Estimate placement parasitics



placement-based parasitics
approximate future routed wires

```
# One set_layer_rc call per layer.  
# -layer takes one name, not a range.  
# Tcl foreach can reuse R/C values  
# across groups of layers.  
# E.g. M1-M4 and M6-M7, separately.  
# Override missing or unsuitable  
# LEF values only.  
# R: kohm/um; C: fF/um in Nangate45.  
set_layer_rc -layer metal3 \  
    -resistance 5e-3 \  
    -capacitance 5e-2  
# set_wire_rc chooses signal and  
# clock layers for wire estimates.  
set_wire_rc -signal -layer metal3  
set_wire_rc -clock -layer metal6  
# Use placed pin locations, not  
# detailed routed wires.  
estimate_parasitics -placement
```

How repair_design fixes violations

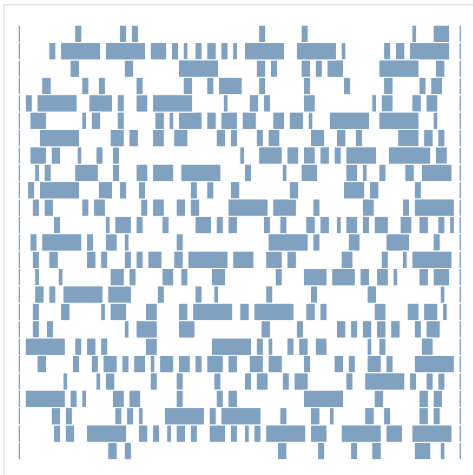
Violation	Problem	Typical fix
Slow transition	Signal edge exceeds the slew limit	Insert buffers or resize the driver
High fanout	One net drives too many pins	Build a buffer tree for the loads
Long wire	Wire RC adds too much delay	Insert buffers along the wire

Estimate with `-placement` before repair and again after CTS.
After global routing, re-estimate with `-global_routing`.

Placement engines

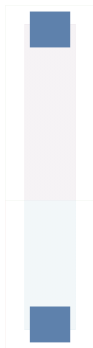
- ▶ Supports mixed cell heights (1x–4x)
- ▶ Supports fence regions and fragmented rows
- ▶ Supports placement padding and filler insertion
- ▶ Two legalization engines:
 - ▶ **Negotiation legalizer (default):** handles dense placement, but is slower; uses diamond search for remaining corner cases.
 - ▶ **Diamond search (legacy):** faster search from each cell's starting position. Enable with `-use_diamond_legalizer`.

Detail placement

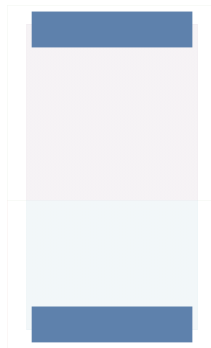


```
# Reserve space beside each cell.
set_placement_padding -global \
    -left 1 -right 1
# Move cells onto legal row sites.
# Resolve overlaps after repair.
# Cap movement to 10 um in each axis.
detailed_placement \
    -max_displacement 10
# Check for placement violations.
check_placement -verbose
```

Nangate45 filler cells in GDS



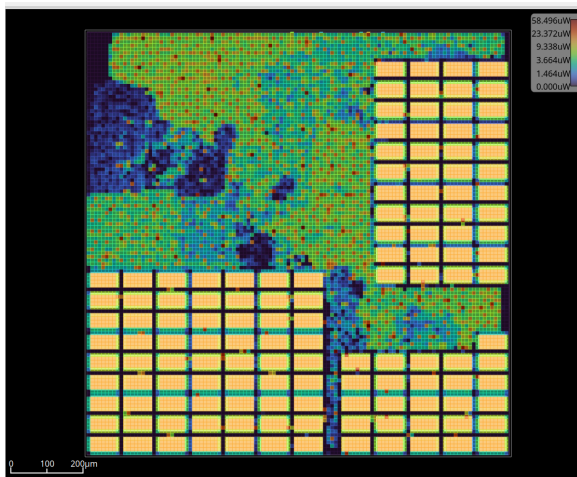
FILLCELL_X1: $0.19 \times 1.4 \mu\text{m}$



FILLCELL_X4: $0.76 \times 1.4 \mu\text{m}$

These cells fill row gaps and carry the VDD/VSS rails.

Heatmaps after placement



- ▶ **Power:** `report_power`;
Heat Maps > Power. Uses activity and estimated RC. CTS and buffers will change it.
- ▶ **Pin density:** Heat Maps > Pin.
- ▶ **Cell density:** Heat Maps > Placement.